

Ministry of Education and Science of the Republic of Kazakhstan
Astana IT University

REPORT

about the implementation of the internship program

Practice base: TOO «PetroSkills Pro»
Student: Mukhtarova Kamilla
Educational program: Software Engineering
Group: SE-2309

Enterprise practice supervisor: Askhat Kuzhakhmetov
University practice supervisor: Elvira Aitmukhanbetova

Astana 2026

Contents

1. Introduction	3
1.1. Place of Practice	3
1.2. Goals and Objectives	3
1.3. Expected Results of Industrial Practice	4
2. Main Part	5
2.1. Theoretical Part	5
2.1.1. General Information About the Company	5
2.1.2. Organizational Structure of the Department	5
2.1.3. Overview of Key Technologies	5
2.1.4. Comparison of LLM Models	6
2.2. Practical Part	6
2.2.1. Project Overview: Sana LMS Platform	7
2.2.2. Research and Analysis of AI-Chat Architecture	7
2.2.3. Design of AI Assistant for LMS	7
2.2.4. Implementation of AI Assistant (MVP)	8
2.2.5. Integration with Sana Platform	8
2.2.6. Technologies Used	9
2.2.7. Challenges and Solutions	9
3. Conclusion	10
4. Reflection	11
5. References	12
6. Practice Task	13
7. Appendices	14

1. Introduction

Industrial practice is an essential part of the educational process aimed at developing practical skills in system analysis, backend development, and AI integration within a real-world project environment. This report presents the results of the industrial practice completed as part of the Software Engineering educational program at Astana IT University.

The practice was focused on participating in the development of a modern Learning Management System (LMS) called Sana, which is designed for corporate training and employee development. The platform integrates modules such as course management, training requests, certification tracking, and learning analytics, providing a complete ecosystem for employee development [2].

During the internship, special attention was given to the design and development of an AI assistant based on Large Language Models (LLM), integrated into the LMS platform. The assistant is intended to support users by providing guidance, answering questions, and simplifying interaction with the system [6].

1.1. Place of Practice

The industrial practice was completed remotely in collaboration with PetroSkills Pro LLP, which is involved in the development of a startup project - AI-LMS, an intelligent learning management system for corporate training.

The project is developed in cooperation with the company Nomad Speakers, which specializes in organizing corporate training, business events, and implementing digital learning solutions in Kazakhstan.

The Sana platform represents a comprehensive LMS system designed to support corporate education processes, improve employee competencies, and provide scalable and flexible learning solutions using modern technologies.

1.2. Goals and Objectives

The main goal of the industrial practice was to gain practical experience in software development, with a focus on backend systems, system architecture, and AI integration within a corporate LMS platform.

The key objectives of the practice included:

- studying the architecture and functionality of the Sana LMS platform
- analyzing AI-chat architectures based on the Encore framework [1]

- comparing modern LLM models (Gemini, GPT-4o, Claude) for integration into the system [5, 6]
- designing an AI assistant adapted for LMS use cases
- developing system prompts for user-friendly AI interaction
- participating in the development of backend API endpoints for AI communication
- integrating AI capabilities into the LMS environment
- improving skills in backend and frontend development [2]
- working collaboratively within a development team

1.3. Expected Results of Industrial Practice

As a result of the industrial practice, it was expected to develop both technical and professional competencies in software engineering, including system analysis, architecture design, and AI integration.

The expected outcomes included:

- development of a functional LMS platform prototype
- implementation of an AI assistant integrated into the system
- development of a functional MVP AI assistant capable of generating responses to user queries in real time
- gaining experience in working with modern technologies such as Go (Encore), Next.js, and LLM-based systems [1, 2, 3]
- understanding real-world software development processes and teamwork

The practice also aimed to enhance problem-solving skills, improve technical knowledge, and prepare for future professional activities in the field of software engineering.

2. Main Part

2.1. Theoretical Part

2.1.1. General Information About the Company

PetroSkills Pro LLP is a Kazakhstan-based company developing an AI-powered Learning Management System (AI-LMS) for corporate training. The project is carried out in cooperation with Nomad Speakers, a company that specializes in organizing corporate training events and implementing digital learning solutions across Kazakhstan.

The Sana LMS platform serves as the core product — a comprehensive system for managing corporate education, tracking employee progress, handling certifications, and enabling data-driven learning analytics.

Learning Management Systems (LMS) have become a fundamental tool in corporate training worldwide. Modern LMS platforms combine traditional course delivery with advanced analytics, automated certification tracking, and increasingly, AI-driven personalization [5]. The integration of AI assistants into LMS platforms represents an emerging trend, enabling real-time user support and context-aware guidance without human intervention.

2.1.2. Organizational Structure of the Department

The development team at PetroSkills Pro is organized into functional sub-teams following an Agile methodology. The team structure consists of the following roles:

Role	Responsibilities
Project Manager	Sprint planning, stakeholder communication, delivery coordination
Backend Engineers (Go)	API development, business logic, database management, AI integration
Frontend Engineers (Next.js)	UI development, component library, API consumption
System Analyst / Intern	Requirements analysis, user flows, architecture documentation
QA Engineer	Testing, bug reporting, quality assurance

During the practice, the intern worked within the backend and AI integration sub-team, collaborating closely with the frontend team for full-stack feature delivery. Communication was conducted via standard remote development tools (Gitlab, Slack), with regular check-ins and code reviews.

2.1.3. Overview of Key Technologies

The Sana LMS platform is built using a modern technology stack. Below is an overview of the key technologies used in the project:

Technology	Purpose	Reference
Go (Encore framework)	Backend API development, service orchestration	[1]
Next.js (React)	Frontend UI, server-side rendering, routing	[2]
Gemini API (Google)	LLM-based AI assistant responses	[3, 4]
Python	Auxiliary scripts for routing extraction and data processing	
Git / VS Code	Version control and development environment	

The Encore framework [1] provides a type-safe, cloud-native approach to building backend services in Go. It handles service discovery, API routing, and authentication out of the box, significantly reducing boilerplate code and allowing developers to focus on business logic.

2.1.4. Comparison of LLM Models

Prior to selecting the AI model for the project, a comparative analysis of the leading Large Language Models was conducted. The evaluation focused on four criteria: response quality, processing speed, cost efficiency, and suitability for business logic scenarios [5, 6].

Criterion	Gemini (Google)	GPT-4o (OpenAI)	Claude (Anthropic)
Response Quality	High	Very High	Very High
Processing Speed	Fast	Medium	Medium
Cost Efficiency	High (free tier)	Medium	Medium
Business Logic Support	Good	Excellent	Excellent
API Documentation	Comprehensive	Comprehensive	Comprehensive
Selected for MVP	Yes	No	No

As a result of this analysis, Gemini was selected as the primary model for MVP implementation due to its balance between performance and cost efficiency. The availability of a free tier for development purposes also made it the most practical choice for a startup environment [3, 4].

2.2. Practical Part

During the industrial practice, a significant part of the work was focused on the research, design, and implementation of an AI assistant integrated into the Sana LMS platform.

At the initial stage, participation in the MVP development was carried out in the role of a system analyst. Responsibilities included designing user flows, defining interface logic (notifications, attendance tracking, empty states), and reviewing frontend code for security aspects such as proper use of environment variables. Additionally, contributions were made to documentation and intermediate presentations were prepared to support the development process.

2.2.1. Project Overview: Sana LMS Platform

Sana LMS is a modern corporate learning management system built to support employee training and development. The platform includes the following key modules: course management, training requests, certification tracking, employee progress analytics, notifications, and external learning (SCORM). The system supports multiple user roles - employee, HR manager, and administrator - each with distinct access rights and functionality.

2.2.2. Research and Analysis of AI-Chat Architecture

Further work was focused on analyzing the possibility of integrating an AI assistant into the LMS platform. The main objective of the assistant is to support users by answering questions, explaining system functionality, and providing personalized recommendations for learning.

The architecture of existing AI-based systems was analyzed, including the ai-chat project built on the Encore framework [1]. Special attention was given to understanding the interaction between components such as chat orchestration, LLM services, and provider layers.

Additionally, a comparative analysis of modern Large Language Models was conducted, including Claude, GPT-4o, and Gemini [5, 6]. The evaluation was based on key criteria such as response quality, speed, cost, and the ability to handle business logic scenarios (see Section 2.1.4). As a result of this analysis, Gemini was selected as the primary model for MVP implementation.

2.2.3. Design of AI Assistant for LMS

Based on the research, a simplified architecture of the AI assistant tailored to LMS requirements was designed. The architecture follows a request-response model:

Next.js → **/assistant/chat** → **Backend (Encore, Go)** → **Gemini** → **Response**

In this architecture:

- the frontend (Next.js) sends user queries to the backend [2]
- the backend processes the request and enriches it with business context (such as user role, course data, and learning progress)
- the request is forwarded to the Gemini model via API [3]
- the generated response is returned to the user through the chat interface

A separate assistant service was designed to handle AI-related logic, including prompt processing, context preparation, and communication with the LLM. This separation of concerns ensures modularity and ease of future model replacement.

2.2.4. Implementation of AI Assistant (MVP)

At the implementation stage, a working MVP of the AI assistant was developed. The backend includes a fully functional API endpoint (`/assistant/chat`) implemented using Go and the Encore framework [1]. This endpoint processes user input, enriches it with contextual data (such as user role, courses, certificates, and employee information), and sends a structured prompt to the Gemini model [3].

The AI assistant uses role-based logic, adapting responses depending on the user type (employee, HR, admin), which improves the relevance and usefulness of responses. The integration with the Gemini API was completed, enabling real-time generation of responses based on user queries.

On the frontend side, a chat interface was implemented using Next.js [2] and integrated into the Sana LMS interface. The UI allows users to send messages and receive responses in real time via HTTP requests to the backend.

The developed MVP was successfully tested in a local development environment, where the frontend and backend were run simultaneously. The AI assistant correctly processed user requests and returned generated responses through the integrated chat interface.

2.2.5. Integration with Sana Platform

In addition to the AI assistant, a Python script was developed to automatically parse the Next.js routing structure. This script extracts routes, functions, and imports from the project, which helps the AI assistant better understand the application structure and provide more relevant responses.

This approach improves the contextual awareness of the assistant and supports more accurate answers related to system navigation and functionality. A reusable React component (AiChat) was created using Next.js [2], including an input field, a send button, and a response display area. It communicates with the backend via HTTP POST requests to the /assistant/chat endpoint.

The chat interface was embedded into an existing LMS page (SCORM module page), ensuring consistency with the overall UI design of the platform.

2.2.6. Technologies Used

During the practice, the following technologies were applied. For a structured overview with references, see the technology table in Section 2.1.3.

The primary tools and frameworks used were:

- Go (Encore) - backend development and API routing [1]
- Next.js - frontend development and UI components [2]
- Gemini API - integration of AI capabilities [3]
- Google Cloud Vertex AI - cloud infrastructure for AI model serving [4]
- Python - auxiliary scripts for data processing
- Git / VS Code - development and collaboration tools

2.2.7. Challenges and Solutions

During the development process, several challenges were encountered and resolved:

Challenge	Solution
Understanding AI architecture - initially difficult to understand how LLM-based systems are structured.	Resolved through analysis of existing solutions such as ai-chat built on Encore [1], and study of official LLM documentation [5, 6].
Choosing the appropriate LLM model - different models offered different trade-offs in quality, speed, and cost.	A structured comparative analysis was conducted across Gemini, GPT-4o, and Claude, resulting in Gemini selection for MVP [3].
Working with API integration - difficulties encountered when configuring and testing API requests.	Resolved through additional debugging, testing in Encore's built-in API Explorer, and referencing API documentation [3, 4].
Designing effective prompts - prompts needed to be structured for consistent and accurate AI responses.	Iterative prompt engineering was applied, with role-based context injection to improve relevance for different user types.

These challenges contributed to gaining deeper practical knowledge and significantly improved problem-solving skills in real-world software development.

3. Conclusion

The industrial practice provided valuable experience in applying theoretical knowledge to a real-world software engineering project. During the internship, the development of a corporate Learning Management System (Sana) was carried out with contributions to both analytical and technical aspects of the project.

One of the key outcomes of the practice was the successful development of a working MVP AI assistant integrated into the Sana LMS platform. The assistant supports real-time interaction with users, processes contextual information from the system, and generates intelligent responses using a Large Language Model (Gemini) [3, 4].

The implementation includes both backend and frontend components, forming a complete end-to-end feature within the LMS. The backend API endpoint (/assistant/chat) was built using Go and the Encore framework [1], while the frontend chat component was developed in Next.js [2]. This allowed the assistant to be directly used inside the platform through a chat interface.

In addition, practical experience was gained working with modern technologies such as Go (Encore), Next.js, and LLM-based APIs. Skills in system analysis, architecture design, and collaborative development within a team environment were also strengthened.

Overall, the practice significantly enhanced understanding of building scalable systems and integrating AI into real-world applications. It also improved the ability to solve complex problems, work with new technologies, and adapt to project requirements. The knowledge and experience gained will be valuable for future professional development in software engineering, particularly in backend development and AI-driven systems.

4. Reflection

During the industrial practice, valuable experience was gained that significantly expanded understanding of software engineering and real-world project development.

At the beginning of the internship, one of the main challenges was understanding the architecture of a complex system and how its components interact. Working on the Sana LMS project helped develop system thinking and understand how frontend, backend, and AI components are integrated within a single application. The Encore framework [1] proved to be a powerful tool that simplified much of the infrastructure complexity.

A particularly important part of the experience was working with AI technologies. Initially, practical experience with Large Language Models was limited, but during the practice different models were analyzed, their capabilities were compared, and the most suitable one for the project was selected (see Section 2.1.4). System prompts were also designed, and it became clear how prompt structure directly affects the quality and consistency of AI responses [5, 6].

Another valuable aspect was working with backend-related tasks and external APIs. Involvement included designing the interaction flow of the AI assistant and implementing the backend endpoint for communication with the LLM [3]. Although this process included challenges related to API integration and system architecture, they were overcome through testing, debugging, and continuous learning.

A particularly valuable achievement was the implementation of a full-stack AI feature, including both backend logic and frontend integration. This allowed for a better understanding of how different system layers interact in real-world applications.

Overall, this practice helped to become more confident in working with complex systems and strengthened practical skills in AI integration and backend development. It also helped identify key areas for further growth, particularly in designing scalable systems and working with AI-driven solutions. This experience confirmed interest in software engineering and motivated continuing development in the field of AI-based applications.

5. References

1. Encore Documentation. Available at: <https://encore.dev/docs>
2. Next.js Documentation. Available at: <https://nextjs.org/docs>
3. Google Gemini API Documentation. Available at: <https://ai.google.dev>
4. Google Cloud Vertex AI Documentation. Available at: <https://cloud.google.com/vertex-ai>
5. OpenAI. Introduction to Large Language Models. Available at: <https://platform.openai.com/docs>
6. Anthropic Claude Documentation. Available at: <https://docs.anthropic.com>
7. Encore AI Chat Example Repository. Available at: <https://github.com/encoredev/examples>

6. Practice Task

During the industrial practice, tasks were assigned related to the development of a Learning Management System (Sana) with a focus on system analysis, backend development, and AI integration.

Main responsibilities included:

- analyzing the architecture and functionality of the Sana LMS platform
- participating in MVP development as a system analyst
- designing user flows and interface logic (notifications, attendance tracking, empty states)
- reviewing frontend code for basic security practices (environment variables usage)
- researching AI assistant integration into LMS systems
- analyzing AI-chat architecture based on the Encore framework [1]
- comparing modern Large Language Models: Gemini, GPT-4o, and Claude [5, 6]
- designing the architecture of an AI assistant for LMS
- developing system prompts for AI interaction
- implementing the backend API endpoint (/assistant/chat) for AI communication using Go (Encore) [1]
- integrating the Gemini API into the backend service [3]
- building context-aware logic using user data (role, courses, certificates, employee info)
- developing a frontend chat component using Next.js [2] and integrating it into the LMS interface
- testing the full interaction flow between frontend, backend, and AI model
- debugging API integration and improving response quality

These tasks allowed gaining both analytical and practical experience in software development and AI system design.

7. Appendices

Appendix A - LMS Interface

The following screenshot shows the main Clients management page of the Sana LMS platform, demonstrating the administrative interface with sidebar navigation, client organization management, and status tracking.

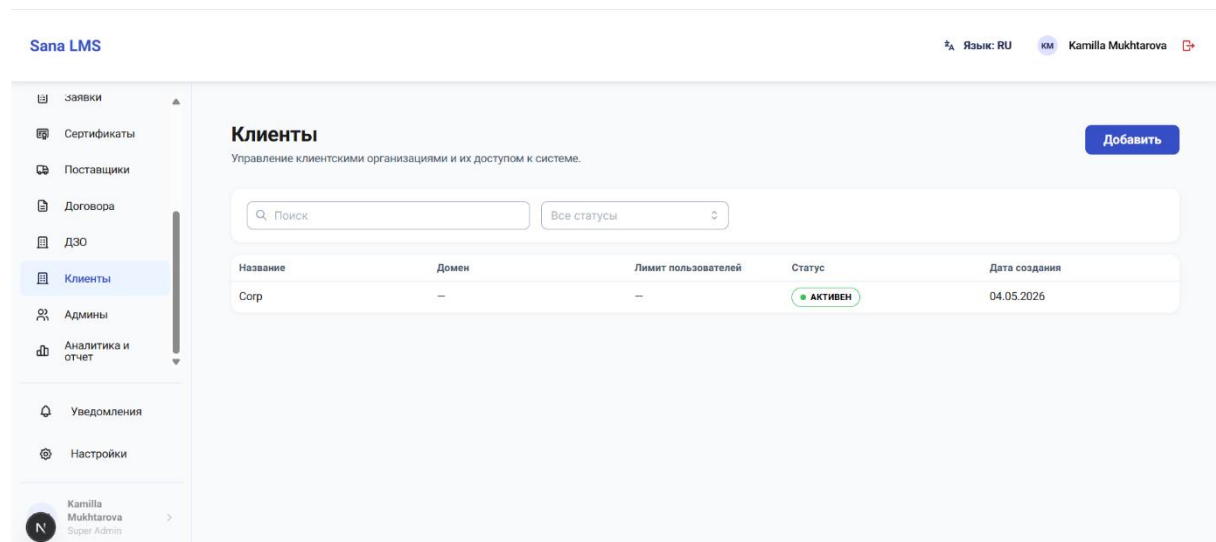


Figure 1. Sana LMS administrative interface — Clients module

Appendix B - AI Assistant UI

The screenshot below demonstrates the AI Assistant chat interface embedded within the SCORM Courses page of the Sana LMS. The interface includes a message input field and a Send button, integrated seamlessly into the existing LMS design.

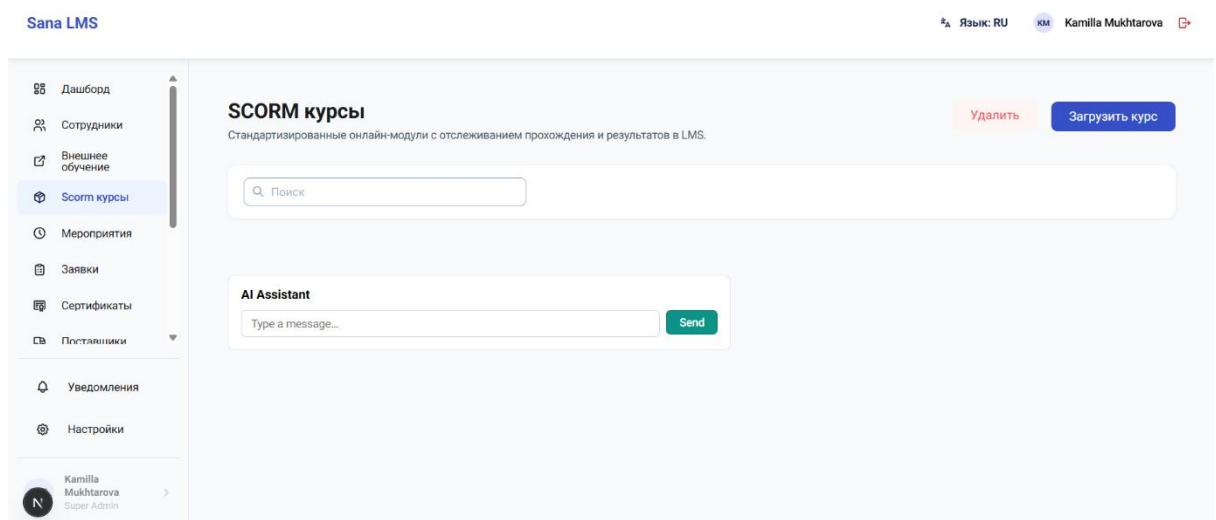


Figure 2. AI Assistant UI embedded in the SCORM module page

Appendix C - Backend API Testing

The following screenshot shows the Encore API Explorer used for testing the /assistant/chat endpoint. The left panel displays the API structure, the center panel shows the request editor with a sample message, and the right panel shows execution traces for various service calls.

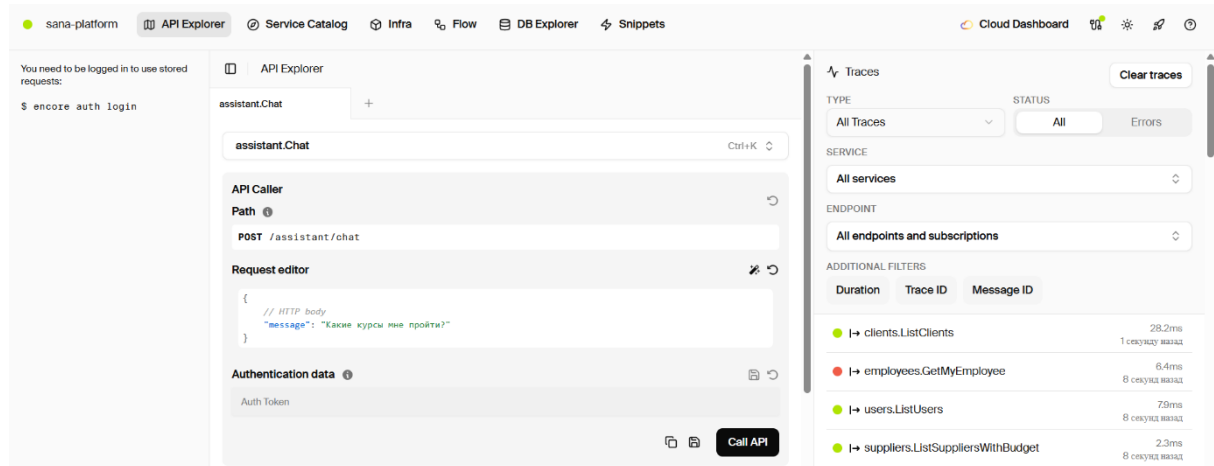


Figure 3. Backend API testing via Encore API Explorer

Appendix D - Code Implementation

Below are the key code fragments from the AI assistant implementation. The first fragment shows the main assistant.go file with the Chat endpoint, and the second shows the Gemini client implementation in gemini.go.

Listing 1. assistant.go - Chat endpoint (excerpt)

```
sana-platform > assistant > assistant.go 1 of 2 | 1 of 3 Files > Undo File Keep File Ctrl+Z
1 package assistant
2
3 import (
4     "context"
5     "fmt"
6     "strings"
7
8     "encore.app/auth/authhandler"
9     "encore.app/courses/events"
10    "encore.app/learning/certificates"
11    "encore.app/orgstructure/employees"
12    "encore.dev/beta/auth"
13    "encore.dev/beta/errs"
14 )
15
16 // — DATABASE —
17
18 // DB section intentionally left empty for skeleton stage.
19
20 // — ENDPOINTS —
21
22 var gemini = newGeminiClient()
23
24 // Chat sends a message to the AI assistant and returns its reply.
25 //
26 //encore:api public method=POST path=/assistant/chat
27 func Chat(ctx context.Context, req *ChatRequest) (*ChatResponse, error) {
28     ad, _ := getAuthData()
29
30     userContext := buildContext(ctx, ad, req.Message)
31     role := "anonymous"
32     if ad != nil {
```

```
package assistant

import (
    "context"
    "fmt"
    "strings"
    "encore.app/auth/authhandler"
    "encore.app/courses/events"
    "encore.app/learning/certificates"
    "encore.app/orgstructure/employees"
    "encore.dev/beta/auth"
    "encore.dev/beta/errs"
)

var gemini = newGeminiClient()

// Chat sends a message to the AI assistant and returns its reply.
//encore:api public method=POST path=/assistant/chat
func Chat(ctx context.Context, req *ChatRequest) (*ChatResponse, error) {
    ad, _ := getAuthData()
    userContext := buildContext(ctx, ad, req.Message)
    role := "anonymous"
    if ad != nil {
        role = string(ad.Role)
    }
    prompt := buildPrompt(userContext, role, req.Message)
    reply, err := gemini.Generate(ctx, prompt)
    if err != nil {
        return nil, errs.WrapCode(err, errs.Internal, "gemini error")
    }
    return &ChatResponse{Message: reply}, nil
}
```

Listing 2. gemini.go - Gemini client (excerpt)

```
25
26 type geminiClient struct {
27     apiKey string
28     endpoint string
29     http *http.Client
30 }
31
32 func newGeminiClient() *geminiClient {
33     return &geminiClient{
34         apiKey: secrets.GeminiAPIKey,
35         endpoint: geminiEndpoint,
36         http: defaultHTTPClient,
37     }
38 }
39
40 // — GEMINI REST TYPES —
41
42 type geminiRequest struct {
43     Contents []geminiContent `json:"contents"`
44 }
45
46 type geminiContent struct {
47     Parts []geminiPart `json:"parts"`
48 }
49
50 type geminiPart struct {
51     Text string `json:"text"`
52 }
53
54 type geminiResponse struct {
55     Candidates []geminiCandidate `json:"candidates"`
56     Error *geminiError `json:"error,omitempty"`
57 }
```

```
type geminiClient struct {
    apiKey string
    endpoint string
    http *http.Client
}

func newGeminiClient() *geminiClient {
    return &geminiClient{
        apiKey: secrets.GeminiAPIKey,
        endpoint: geminiEndpoint,
        http: defaultHTTPClient,
    }
}

type geminiRequest struct {
    Contents []geminiContent `json:"contents"`
}

type geminiContent struct {
    Parts []geminiPart `json:"parts"`
}

type geminiPart struct {
    Text string `json:"text"`
}
```

The code demonstrates the clean separation of concerns between the endpoint handler (assistant.go) and the Gemini API client (gemini.go). The role-based context injection in the Chat function ensures that AI responses are tailored to the specific user type (employee, HR manager, or administrator) [1, 3].